Function Calling with Generative AI: A Comprehensive Guide

Introduction

Function calling is a powerful capability that allows generative AI models to interact with external systems by producing structured data outputs. This feature bridges the gap between natural language understanding and programmatic actions, enabling AI to not just comprehend requests but to translate them into precise, executable functions.

This handout will provide you with a thorough understanding of function calling, including its purpose, implementation details, and practical applications. By the end, you'll be equipped with the knowledge to leverage this feature in your own projects, regardless of your current technical expertise.

What is Function Calling?

At its core, function calling allows generative AI models to:

1. Interpret natural language requests from users
2. Determine which function would best fulfill that request
3. Generate structured data outputs (with the appropriate parameters)
4. Enable developers to execute these functions and return results to the model

Instead of the AI model simply responding with text, it can identify when a request would be better served by calling a specific function and will generate the appropriate function call with the necessary parameters.

Why Function Calling Matters

Function calling solves several critical challenges:

- **Structured Data Extraction**: It converts unstructured natural language into structured data.
- **System Integration**: It connects AI models to external tools, databases, and APIs.
- **Accuracy**: It ensures precise interpretation of requests that require specific actions.
- **Up-to-date Information**: It allows the AI to access real-time data beyond its training cutoff.

How Function Calling Works

The process involves three main steps:

1. **Function Definition**: You define functions that the AI can call, including detailed descriptions.
2. **Model Configuration**: You configure the AI to recognize these functions as available tools.

3. **Request Processing**: When a user makes a request, the model determines whether to respond with text or call a function.

Let's explore each of these steps in detail.

1. Function Definition

When defining functions for the AI to call, you need to provide:

- A meaningful function name
- A detailed description of what the function does
- Parameters with types and descriptions
- Return value information

The more detail you provide, the better the model can determine when and how to use the function.

Here's an example of a well-defined function:

```
def set_light_values(brightness: int, color_temp: str) -> dict[str, int | 
str]:
    """Set the brightness and color temperature of a room light.

    Args:
        brightness: Light level from 0 to 100. Zero is off and 100 is full 
brightness
        color_temp: Color temperature of the light fixture, which can be 
`daylight`, `cool` or `warm`.

    Returns:
        A dictionary containing the set brightness and color temperature.
    """
    # Function implementation
    return {
        "brightness": brightness,
        "colorTemperature": color_temp
    }
```

Notice the thorough docstring that explains: - The function's purpose - Each parameter's meaning and acceptable values - The structure of the return value

These details help the AI model understand exactly when this function should be called and how to populate its parameters correctly.

2. Model Configuration

Once you've defined your functions, you need to configure the model to recognize them as available tools. This typically involves creating a configuration object that includes your functions.

Here's how you might configure a model to use the light control function:

```python
from google.genai import types

# Using Python objects for configuration
config = types.GenerateContentConfig(tools=[set_light_values])

# Or using a dictionary
config = {
    'tools': [set_light_values]
}
```

This configuration tells the model that `set_light_values` is available as a tool it can use when processing requests.

3. Request Processing

When the model receives a user request, it analyzes the content to determine whether it should: - Respond with natural language text - Call one of the defined functions

If the model determines a function call is appropriate, it will generate a structured output containing: - The name of the function to call - Values for each required parameter

For example, if a user says: “Turn the lights down to a movie watching level”

The model might generate a function call like:

```python
set_light_values(brightness=30, color_temp="warm")
```

The model has interpreted that “movie watching level” likely means dimmed lights (30% brightness) with a warm color temperature.

Function Calling Modes

The behavior of function calling can be controlled through different modes:

AUTO Mode (Default)

In AUTO mode, the model decides whether to call a function or provide a text response based on its understanding of the request.

```python
# AUTO is the default mode
config = {
    'tools': [set_light_values]
}
```

This is ideal for general-purpose assistants that need to handle a mix of conversational and action-oriented requests.

ANY Mode

In ANY mode, the model is constrained to always predict a function call, never a text response.

```
config = {
    'tools': [set_light_values],
    'tool_config': {
        'function_calling_config': {
            'mode': 'any'
        }
    }
}
```

This is useful when you know the user's input requires an action and you want to ensure the model calls a function.

NONE Mode

In NONE mode, the model will never predict a function call.

```
config = {
    'tool_config': {
        'function_calling_config': {
            'mode': 'none'
        }
    }
}
```

This is useful when you want the model to provide only text responses, even if functions are defined.

Automatic vs. Manual Function Execution

There are two approaches to handling function execution:

Automatic Function Calling

With automatic function calling, the SDK handles the process of: 1. Receiving the function call from the model 2. Executing the function with the provided parameters 3. Returning the result to the model

```
# Enable automatic function calling (default behavior)
config = {
    'tools': [set_light_values]
}

chat = client.chats.create(model='gemini-2.0-flash', config=config)
response = chat.send_message('Turn the lights down to a movie watching
level')
print(response.text)
```

This approach simplifies development by automating the entire process.

Manual Function Calling

With manual function calling, you handle each step of the process:

```
# Disable automatic function calling
config = {
    'tools': [set_light_values],
    'automatic_function_calling': {'disable': True}
}

chat = client.chats.create(model='gemini-2.0-flash', config=config)
response = chat.send_message('Turn the lights down to a movie watching
level')

# Extract function call details
for fn in response.function_calls:
    name = fn.name
    args = fn.args

    # Execute the function manually
    if name == "set_light_values":
        result = set_light_values(**args)

        # Send the result back to the model
        response = chat.send_message(function_response={
            'name': name,
            'response': result
        })

        print(response.text)
```

This approach gives you more control over the execution process, allowing you to: - Add custom validation - Handle errors gracefully - Log function calls - Apply business logic between function calls

Advanced Feature: Parallel Function Calling

Sometimes a single user request might require multiple actions. Parallel function calling allows the model to call multiple functions in response to a single request.

Consider a scenario where you want to set up a party environment:

```
def power_disco_ball(power: bool) -> bool:
    """Powers the spinning disco ball."""
    # Implementation
    return True

def start_music(energetic: bool, loud: bool) -> str:
    """Play some music matching the specified parameters.
```

```
    Args:
      energetic: Whether the music is energetic or not.
      loud: Whether the music is loud or not.

    Returns: The name of the song being played.
    """
    # Implementation
    return "Never gonna give you up."

def dim_lights(brightness: float) -> bool:
    """Dim the lights.

    Args:
      brightness: The brightness of the lights, 0.0 is off, 1.0 is full.
    """
    # Implementation
    return True

# Configure the model with all three functions
house_fns = [power_disco_ball, start_music, dim_lights]
config = {
    'tools': house_fns
}

chat = client.chats.create(model='gemini-2.0-flash', config=config)
response = chat.send_message('Turn this place into a party!')

# With automatic function calling enabled, the SDK will:
# 1. Call power_disco_ball(power=True)
# 2. Call start_music(energetic=True, loud=True)
# 3. Call dim_lights(brightness=0.3)
# 4. Return the results to the model
# 5. Generate a final response

print(response.text)
# Output: "Alright, I've turned on the disco ball, started playing..."
```

In this example, the model interpreted “Turn this place into a party!” as requiring multiple actions and generated calls to all three functions.

Function Declaration and Type Mapping

While the examples above use Python functions directly, you can also define functions more explicitly using declaration objects. This gives you more control over how functions are presented to the model.

```
from google.genai import types

# Direct function declaration
```

```
tool = types.Tool(function_declarations=[
    types.FunctionDeclaration(
        name="multiply",
        description="Returns a * b.",
        parameters=types.Schema(
            properties={
                'a': types.Schema(type='NUMBER'),
                'b': types.Schema(type='NUMBER'),
            },
            type='OBJECT',
        ),
    )
])

# Or extract from a callable
def multiply(a: float, b: float):
    """Returns a * b."""
    return a * b

fn_decl = types.FunctionDeclaration.from_callable(callable=multiply,
client=client)
```

The SDK converts Python type annotations to formats the API understands, but there are limitations on what types are supported: - Basic types: int, float, bool, str - Container types: list, dict - Complex nested structures may require manual declaration

Real-World Examples

To help illustrate how function calling can be applied in practice, let's explore a few real-world examples:

Example 1: Weather Information System

```
def get_weather(location: str, unit: str = "celsius") -> dict:
    """Get current weather information for a location.

    Args:
        location: City name or postal code
        unit: Temperature unit, either 'celsius' or 'fahrenheit'

    Returns:
        Weather information including temperature, conditions, and humidity
    """
    # In a real implementation, this would call a weather API
    # For demonstration, we'll return mock data
    return {
        "location": location,
        "temperature": 22.5 if unit == "celsius" else 72.5,
        "conditions": "Partly cloudy",
        "humidity": 65
```

```
    }

# User: "What's the weather like in Chicago?"
# Model would call: get_weather(location="Chicago", unit="celsius")
```

Example 2: E-commerce Order System

```
def search_products(query: str, category: str = None, max_price: float =
None) -> list:
    """Search for products in the catalog.

    Args:
        query: Search terms
        category: Optional category to filter by
        max_price: Optional maximum price to filter by

    Returns:
        List of matching products
    """
    # Implementation would query product database
    pass

def add_to_cart(product_id: str, quantity: int = 1) -> dict:
    """Add a product to the user's shopping cart.

    Args:
        product_id: Unique identifier for the product
        quantity: Number of items to add

    Returns:
        Updated cart information
    """
    # Implementation would update user's cart
    pass

# User: "Find me headphones under $100"
# Model would call: search_products(query="headphones", max_price=100)

# User: "Add the Sony ones to my cart"
# Model would call: add_to_cart(product_id="sony-wh1000xm4", quantity=1)
```

Example 3: Calendar Management

```
def check_availability(date: str, start_time: str, end_time: str) -> bool:
    """Check if a time slot is available on the calendar.

    Args:
        date: Date in YYYY-MM-DD format
        start_time: Start time in 24-hour format (HH:MM)
        end_time: End time in 24-hour format (HH:MM)
```

```
    Returns:
        True if the slot is available, False otherwise
    """
    # Implementation would check calendar
    pass

def schedule_meeting(title: str, date: str, start_time: str, end_time: str,
                     attendees: list[str] = None) -> dict:
    """Schedule a meeting on the calendar.

    Args:
        title: Meeting title
        date: Date in YYYY-MM-DD format
        start_time: Start time in 24-hour format (HH:MM)
        end_time: End time in 24-hour format (HH:MM)
        attendees: Optional list of email addresses for attendees

    Returns:
        Meeting details including unique identifier
    """
    # Implementation would create calendar event
    pass

# User: "Schedule a team meeting next Monday at 2pm for an hour"
# Model would:
# 1. Calculate next Monday's date (e.g., "2025-03-31")
# 2. Call: check_availability(date="2025-03-31", start_time="14:00",
end_time="15:00")
# 3. If available, call: schedule_meeting(title="Team Meeting", date="2025-
03-31",
#                                         start_time="14:00",
end_time="15:00")
```

Best Practices for Function Calling

To get the most out of function calling, follow these best practices:

1. Write Clear Function Descriptions

The model relies heavily on function and parameter descriptions to determine when and how to call functions. Be thorough and precise in your descriptions.

Good Example:

```
def book_flight(
    departure_city: str,
    arrival_city: str,
    departure_date: str,
    return_date: str = None,
    num_passengers: int = 1,
```

```
    class_preference: str = "economy"
) -> dict:
    """Book a flight between two cities.

    Args:
        departure_city: City where the flight will depart from
        arrival_city: Destination city
        departure_date: Departure date in YYYY-MM-DD format
        return_date: Optional return date in YYYY-MM-DD format for round
trips
        num_passengers: Number of passengers (default: 1)
        class_preference: Cabin class, one of: "economy", "premium_economy",
"business", "first" (default: "economy")

    Returns:
        Booking information including confirmation code, flight details, and
price
    """
```

2. Handle Errors Gracefully

When a function call fails, provide helpful error messages that the model can use to correct the issue or inform the user.

```
def transfer_money(from_account: str, to_account: str, amount: float) ->
dict:
    """Transfer money between accounts.

    Args:
        from_account: Source account number
        to_account: Destination account number
        amount: Amount to transfer (must be positive)

    Returns:
        Transaction details including confirmation number
    """
    if amount <= 0:
        raise ValueError("Transfer amount must be positive")

    if amount > get_account_balance(from_account):
        raise ValueError("Insufficient funds in source account")

    # Process transfer...
```

3. Choose the Right Function Calling Mode

Select the appropriate function calling mode based on your use case: - Use AUTO for general-purpose assistants - Use ANY when you know a function call is required - Use NONE when you want only text responses

4. Group Related Functions

Organize related functions together to make it easier for the model to select the right one.

```
weather_tools = [get_current_weather, get_forecast, get_weather_alerts]
calendar_tools = [check_availability, schedule_meeting, cancel_meeting]

# Configure based on context
if context == "weather":
    config = {'tools': weather_tools}
elif context == "calendar":
    config = {'tools': calendar_tools}
```

5. Test with Various Phrasings

Users may express the same intent in different ways. Test your function calling setup with various phrasings to ensure it works consistently.

For example, test the light control function with: - “Turn the lights down for a movie” - “I want to watch a film, adjust the lighting please” - “Make it cozy for movie night” - “Dim the lights to 30%”

6. Implement Feedback Loops

Allow the model to refine its function calls based on results or errors.

```
try:
    result = execute_function(fn_name, fn_args)
    response = chat.send_message(function_response={'name': fn_name,
'response': result})
except ValueError as e:
    # Send the error back to the model
    response = chat.send_message(function_response={
        'name': fn_name,
        'error': str(e)
    })
    # The model can now suggest an alternative approach
```

Conclusion

Function calling transforms generative AI from a purely conversational tool into a powerful interface for digital systems. By enabling models to call functions with appropriate parameters based on natural language requests, this feature opens up a wide range of applications where precise actions are required.

As you implement function calling in your own projects, remember that the quality of your function definitions and descriptions directly impacts the model’s ability to use them effectively. Invest time in creating clear, comprehensive definitions, and your AI systems will become significantly more capable and user-friendly.